Práctica. Simulacro de incidente en tu web

Nombre y apellidos: _____

Miércoles 18 de noviembre de 2026 · IC · 4 horas · trabajo individual

Hasta ahora vuestro Wazuh vigilaba registros que yo os daba hechos: el SSH de un servidor que no existe y un portal de nóminas inventado. Esta semana vigila algo vuestro. El martes pusisteis en marcha miweb en vuestro Ubuntu, con nginx delante y un registro de seguridad en JSON. Hoy la vais a atacar desde vuestra Kali, vais a ver el ataque llegar a Wazuh y vais a responder como lo haría un SOC: detectar, valorar, contener, guardar las pruebas y contarlo.

El ataque lo lanza un script, así que todos tendréis más o menos las mismas alertas. Lo que no os va a dar el script es lo que importa: decidir qué alerta es grave y cuál es ruido, cortar el ataque sin tirar la web, conservar las pruebas de forma que sirvan y escribir un informe que se entienda. El jueves, en HE, el ataque lo haréis vosotros a mano, y el miércoles que viene investigaréis en Wazuh lo que hicisteis el jueves.

1. Qué vas a aprender hoy

1. Añadir fuentes de registro nuevas a un agente de Wazuh, en JSON y en el formato de acceso de nginx, sin romper su configuración.
2. Cargar un conjunto de reglas en un fichero propio, separado del de la semana pasada.
3. Explicar cómo funciona una regla de correlación que encadena dos alertas (fuerza bruta seguida de acceso correcto).
4. Hacer el triaje de un ataque real: agrupar alertas, separar ruido y señal, y decidir la gravedad.
5. Contener un ataque web bloqueando una IP en nginx y comprobar que el bloqueo funciona.
6. Conservar evidencias con su huella SHA-256 y una cadena de custodia sencilla.
7. Reconstruir la línea temporal con hora local y UTC, y redactar un informe de incidente.

2. Lo que necesitas

- **miweb funcionando en tu Ubuntu**, como la dejaste el martes en PPS: el servicio `miweb` con gunicorn y nginx escuchando en el puerto 8080. Si no la terminaste, haz antes el despliegue de la práctica de PPS del 17/11 (`despliegue/instalar-servicio.sh`). Sin la web no hay simulacro.
- **Tu Kali con la tarjeta solo-anfitrión** (red 192.168.56.0/24) que conectaste el martes con `red-solo-anfitrion.sh`.
- **Tu Wazuh**: el manager en Docker en `~/wazuh-docker/single-node` y el agente instalado en el propio Ubuntu, el mismo de las prácticas anteriores.

Herramientas: `docker compose`, `jq`, `curl`, `sha256sum` y un editor (`nano` o el que uses).

Descarga el ZIP del laboratorio de la web del curso (IC, Prácticas) en el Ubuntu y descomprímelo. Si tus carpetas se llaman de otra forma, ajusta las rutas:

```
mkdir -p ~/ic
cd ~/ic
unzip ~/Descargas/IC-2026-11-18-simulacro-LAB.zip
cd IC-2026-11-18-simulacro-LAB
ls
```

Descarga también el ZIP en la Kali, desde el navegador de la Kali, y descomprímelo en `~/ic`. En la Kali solo vas a usar `ataque-simulado.sh`.

Lee el `README.md`. Ahí tienes la tabla de ficheros y de direcciones que vas a usar toda la mañana.

3. Comprueba el punto de partida (15 minutos)

En el Ubuntu:

```
systemctl status miweb --no-pager | head -3
curl -s http://127.0.0.1:8080/salud
```

Tiene que salir `active (running)` y `{"estado":"ok"}`. En la Kali:

```
ip -br a
curl -s http://192.168.56.1:8080/salud
```

La Kali tiene que tener una dirección 192.168.56.1xx en una de sus tarjetas y el `curl` tiene que responder lo mismo que en el Ubuntu. Apunta esa IP en la cabecera de la plantilla: es la IP del atacante.

En el Ubuntu, comprueba Wazuh:

```
cd ~/wazuh-docker/single-node
docker compose ps
docker compose exec -T wazuh.manager /var/ossec/bin/agent_control -l
```

`wazuh.manager` tiene que estar en marcha y tu agente `Active`. Apunta su nombre. Rellena la tabla del apartado 1 de la plantilla según vayas haciendo este apartado y los dos siguientes.

4. Conecta las dos fuentes al agente (25 minutos)

El agente de Wazuh solo lee los ficheros que tiene apuntados en su configuración. Los de miweb son nuevos, así que hay que decírselo. Abre `ossec-localfile.xml` y mira los dos bloques:

- El primero lee `/var/log/miweb/seguridad.log` con `log_format` **json**. Así, Wazuh separa él solo cada campo de la línea (`evento`, `usuario`, `ip`, `ruta`, `estado`, `detalle`) y tus reglas pueden usarlos directamente. No hace falta escribir ningún decodificador, al revés que la semana pasada.
- El segundo lee `/var/log/nginx/miweb_access.log` con `log_format` **syslog**, que quiere decir "una línea es un evento". Wazuh reconoce el formato de acceso de nginx con su decodificador de serie `web-accesslog` y le aplica sus reglas de servidores web.

Antes de tocar nada, haz una copia de la configuración del agente:

```
sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.antes-miweb
sudo nano /var/ossec/etc/ossec.conf
```

Baja hasta el final del fichero. Vas a ver que hay **dos** bloques `<ossec_config>`, y por tanto dos líneas `</ossec_config>`. Pega los dos bloques `<localfile>` (solo los bloques, desde `<localfile>` hasta `</localfile>`, con sus comentarios de una línea si quieres) justo antes de la **última** `</ossec_config>`. Guarda y comprueba la configuración antes de reiniciar:

```
sudo /var/ossec/bin/wazuh-logcollector -t
```

Si no sale nada, está bien. Si sale `ERROR: (1226): Error reading XML file`, has dejado una etiqueta mal cerrada o has pegado texto fuera de un bloque. Corrígelo antes de seguir. Si no lo encuentras, vuelve a la copia con `sudo cp /var/ossec/etc/ossec.conf.antes-miweb /var/ossec/etc/ossec.conf` y repite el pegado.

Con la comprobación limpia, reinicia el agente y mira si ha cogido los ficheros:

```
sudo systemctl restart wazuh-agent
sudo grep -E "miweb" /var/ossec/logs/ossec.log | tail -4
```

Tienen que salir dos líneas `Analyzing file:`, una para cada fichero. Si sale `Could not open file`, el fichero no existe con ese nombre: comprueba la ruta con `ls -l /var/log/miweb/ /var/log/nginx/`.

Una pregunta para la plantilla: `/var/log/miweb` es tuyo y tiene permisos 750, y los registros de nginx son de `root`. ¿Por qué el agente puede leerlos igualmente? Mira con `ps -o user,cmd -C wazuh-logcollector` con qué usuario se ejecuta.

5. Sube las reglas de miweb (25 minutos)

Abre `miweb_rules.xml` y léelo entero antes de subirlo. Están todas las reglas, porque la semana pasada ya escribisteis las vuestras y hoy toca usarlas. Fíjate en estas cosas:

- La **100200** es la raíz: recoge cualquier línea decodificada como JSON cuyo campo `app` sea `miweb`. Tiene nivel 0, así que no avisa. Todas las demás cuelgan de ella con `if_sid`.
- La **100203** cuenta accesos fallidos (`if_matched_sid` 100201) desde la misma IP (`same_field ip`). Usamos `same_field` y no `same_source_ip` porque nuestro JSON llama `ip` al campo, no `srcip`.
- La **100204** es nueva: salta con un acceso **correcto** (`if_sid` 100202) si en los cinco minutos anteriores ha saltado una fuerza bruta (`if_matched_sid` 100203) desde la misma IP. Es la diferencia entre "alguien lo ha intentado" y "alguien lo ha conseguido".
- La **100207** busca patrones de inyección SQL en el campo `detalle` de las búsquedas, con una expresión `pcre2`.
- La **100210** tiene nivel 2 y no 0. Lee el comentario: tiene que ver con la 100211.

Sube las reglas. El script no toca el `local_rules.xml` del portal de nóminas: las de miweb van en su propio fichero, `miweb_rules.xml`, dentro de `/var/ossec/etc/rules/`, y Wazuh carga todos los ficheros de esa carpeta.

```
chmod +x subir-reglas.sh
./subir-reglas.sh
```

Tiene que terminar con `OK`. Prueba una línea real de tu web con `wazuh-logtest`. Primero provoca un acceso fallido desde el Ubuntu y copia la última línea del registro:

```
curl -s -o /dev/null -d nombre=lucia -d clave=prueba
http://127.0.0.1:8080/login
tail -n 1 /var/log/miweb/seguridad.log
```

En la terminal de Wazuh:

```
cd ~/wazuh-docker/single-node
docker compose exec wazuh.manager /var/ossec/bin/wazuh-logtest
```

Pega la línea. En la fase 2 tiene que salir `name: 'json'` y en la fase 3 la regla `100201` con nivel 5. Haz lo mismo con la última línea de `/var/log/nginx/miweb_access.log` (necesitas `sudo tail`): tiene que salir el decodificador `web-accesslog` y la regla `31101`, `Web server 400 error code`. Fíjate en que el 401 del acceso fallido aparece en las dos fuentes: la aplicación sabe que era `lucia`; nginx solo sabe que hubo un 401.

Ahora sí, reinicia el manager para que cargue las reglas de verdad:

```
docker compose restart wazuh.manager
```

Espera a que el agente vuelva a estar `Active` (uno o dos minutos) y **un minuto más** antes de lanzar el ataque: si atacas mientras el agente se está reconectando, puedes perder los primeros

eventos.

Mientras esperas, rellena la tabla del apartado 2 de la plantilla.

6. El ataque (20 minutos)

Abre `ataque-simulado.sh` en la Kali y léelo antes de ejecutarlo. Solo usa `curl` contra la dirección de tu web y hace cinco fases:

1. Pide veinte rutas que no existen (`.env`, `.git/HEAD`, `wp-login.php`...), como haría un escáner.
2. Prueba contraseñas contra el usuario `marcos` hasta que acierta.
3. Con la sesión de `marcos`, mete inyecciones SQL en la búsqueda.
4. Abre reservas por número, incluidas las de otros usuarios.
5. Intenta entrar en `/admin`.

Lánzalo desde la carpeta del laboratorio en la Kali:

```
cd ~/ic/IC-2026-11-18-simulacro-LAB
bash ataque-simulado.sh
```

Entre fase y fase te pide que pulses Intro. Antes de pulsar, mira en el Ubuntu cómo crecen los dos registros, en dos terminales:

```
tail -f /var/log/miweb/seguridad.log
sudo tail -f /var/log/nginx/miweb_access.log
```

Al terminar, el script deja en `horas-ataque.txt` la hora de inicio de cada fase, en local y en UTC. Cópialo en el apartado 3 de la plantilla. En un incidente real esto no lo tendrías: el atacante no te dice a qué hora empieza. Hoy lo usamos para comprobar que tu línea temporal cuadra.

7. Detección y triaje (45 minutos)

Ahora te cambias de lado. Olvida que sabes lo que ha hecho el script y mira solo lo que ha visto Wazuh. Saca las alertas de miweb y de nginx a un fichero:

```
mkdir -p ~/ic/simulacro
cd ~/wazuh-docker/single-node
docker compose exec -T wazuh.manager \
    tail -n 5000 /var/ossec/logs/alerts/alerts.json \
    | jq -c 'select(.rule.groups | index("miweb") or index("web"))' \
    > ~/ic/simulacro/alertas.jsonl
wc -l ~/ic/simulacro/alertas.jsonl
```

El filtro se queda con las alertas cuyo grupo incluye `miweb` (tus reglas) o `web` (las reglas de serie de servidores web). Primero, una vista de conjunto: cuántas veces ha saltado cada regla.

```
cd ~/ic/simulacro
jq -r '.rule.id + " " + (.rule.level|tostring) + " " + .rule.description' \
    alertas.jsonl | sort | uniq -c | sort -rn
```

Vas a ver dos familias. Las **311xx y 315xx** son de Wazuh y miran el registro de nginx: códigos 4xx, ráfagas de errores desde la misma IP (31151), patrones de inyección en la URL (31164), rutas sospechosas (31516) y un ataque que ha devuelto 200 (31106). Las **1002xx** son las tuyas y miran lo que sabe la aplicación. Ahora ordénalas en el tiempo:

```
jq -r '[.timestamp, .rule.id, .rule.level, (.data.srcip // .data.ip),
  .rule.description] | @tsv' alertas.jsonl
```

`data.srcip` es la IP que saca nginx; `data.ip` es la del JSON de miweb. El `//` de `jq` quiere decir "si no está la primera, usa la segunda". Y ahora solo lo grave, nivel 10 o más:

```
jq -r 'select(.rule.level >= 10) | [.timestamp, .rule.id,
  .rule.description] | @tsv' alertas.jsonl
```

Busca también las alertas en el Dashboard: en el módulo de búsqueda de amenazas (*Threat Hunting*), pestaña de eventos, filtra con `rule.groups:miweb`. Abre la 100204 y haz una captura con la alerta desplegada.

Rellena la tabla de triaje del apartado 4 de la plantilla. Para cada tipo de alerta decide:

- Si es un **verdadero positivo** (hay un ataque detrás) o ruido. Ojo: la 31101 sale con cualquier 4xx, también con el 401 de alguien que se equivoca de contraseña una vez.
- Su **gravedad**, que no es lo mismo que el nivel de la regla. El nivel lo pone quien escribe la regla sin saber el contexto; la gravedad la pones tú sabiendo qué ha pasado. Un intento de entrar en `/admin` que acaba en 403 es un ataque, pero ha fallado.
- Qué te dice y qué harías.

Contesta en la plantilla: ¿cuál es la alerta más grave del incidente y por qué? ¿Qué cuenta ha quedado comprometida? ¿Qué datos ha podido sacar el atacante con la inyección SQL? Para esto último, mira el campo `detalle` de las búsquedas en `seguridad.log` y piensa qué devuelve esa consulta.

Antes de seguir, sitúa este trabajo en el ciclo de respuesta a incidentes. La guía del NIST (SP 800-61, revisión 3, de 2025) ya no lo describe como cuatro fases fijas, sino dentro de las funciones del marco CSF 2.0: preparar, detectar, responder y recuperar, con mejora continua. Las fases clásicas siguen siendo una buena forma de ordenarse: preparación (lo que hicisteis el martes y en los apartados 4 y 5), detección y análisis (este apartado), contención, erradicación y recuperación (los dos siguientes y la semana que viene) y actividad posterior al incidente (el informe).

8. Contención (25 minutos)

El atacante sigue teniendo acceso: tiene la contraseña de `marcos` y una sesión abierta. Lo primero es cortarle el paso sin tirar la web para los demás. Lo más rápido es bloquear su IP en nginx, que está delante de la aplicación. Edita la configuración del sitio:

```
sudo cp /etc/nginx/sites-available/miweb ~/ic/simulacro/nginx-miweb.antes
sudo nano /etc/nginx/sites-available/miweb
```

Añade una línea `deny` con la IP de tu Kali **antes** de la línea `allow 192.168.56.0/24`. nginx aplica la primera regla que coincide, empezando por arriba, así que si la pones después no sirve de nada. Con la IP 192.168.56.101 quedaría así:

```
allow 127.0.0.1;
deny  192.168.56.101;   # contención incidente 18/11
allow 192.168.56.0/24;
deny  all;
```

Comprueba la configuración y recarga nginx. `reload` aplica los cambios sin cortar las conexiones que haya en curso; si `nginx -t` da error, no recargues:

```
sudo nginx -t
sudo systemctl reload nginx
```

Prueba desde la Kali y desde el Ubuntu:

```
curl -s -o /dev/null -w "%{http_code}\n" http://192.168.56.1:8080/salud
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8080/salud
```

Desde la Kali tiene que salir 403 y desde el Ubuntu 200. Mira ahora qué queda en cada registro después de la petición de la Kali:

```
sudo tail -n 2 /var/log/nginx/miweb_access.log
sudo tail -n 2 /var/log/nginx/miweb_error.log
tail -n 2 /var/log/miweb/seguridad.log
```

En el de acceso de nginx sale el 403 y en el de errores, `access forbidden by rule`. En el de miweb **no sale nada**: la petición ya no llega a la aplicación. Apunta qué consecuencia tiene esto para la vigilancia: a partir de ahora, ¿qué reglas pueden seguir viendo al atacante y cuáles no?

Bloquear una IP es una contención a corto plazo, no una solución. Contesta en la plantilla: ¿qué haría el atacante si cambiara de IP? ¿Qué otras medidas de contención habría que tomar con la cuenta de `marcos`? No las apliques hoy: las verás en el apartado de erradicación del informe.

9. Conserva las evidencias (25 minutos)

Antes de tocar nada más, guarda copia de lo que prueba el incidente. Las copias se hacen con `cp --preserve=timestamps` para no perder la fecha de modificación de los ficheros originales, y se les calcula la huella SHA-256: si alguien cambia un solo byte de una copia, la huella deja de coincidir.

```
mkdir -p ~/ic/simulacro/evidencias
cd ~/ic/simulacro/evidencias
sudo cp --preserve=timestamps /var/log/miweb/seguridad.log .
sudo cp --preserve=timestamps /var/log/nginx/miweb_access.log .
sudo cp --preserve=timestamps /var/log/nginx/miweb_error.log .
cp ../alertas.jsonl .
sudo chown $USER: *
chmod 444 *
sha256sum * | tee SHA256SUMS
date "+%Y-%m-%d %H:%M:%S %z"
```

`chmod 444` deja las copias en solo lectura, para no modificarlas sin querer. Rellena la tabla de evidencias y la cadena de custodia del apartado 6 de la plantilla: quién las recoge, cuándo y dónde quedan. Comprueba al final de la clase que nada ha cambiado:

```
sha256sum -c SHA256SUMS
```

Tienen que salir todas `OK`. Las evidencias que vas a entregar son copias, y lo dices así en el informe. Los registros originales siguen creciendo en el servidor, y por eso las huellas solo valen para las copias.

10. Línea temporal e informe (30 minutos)

Reconstruye la línea temporal del incidente en el apartado 7 de la plantilla, con estas normas:

1. **Hora local y UTC en cada fila.** `seguridad.log` lleva la zona horaria al final (`+01:00`), nginx también (`+0100`) y las alertas de Wazuh salen como las guarda el manager. Comprueba con una alerta concreta si tu manager guarda en UTC o en hora local; si no lo tienes claro, compara con la hora de `horas-ataque.txt`.
2. **Agrupa.** Veinte peticiones de enumeración son una fila: "de tal hora a tal hora, 20 rutas inexistentes desde 192.168.56.101".
3. **Indica la fuente** de cada fila y marca si lo sabes (lo dice un registro) o lo supones (lo deduces).
4. **Cuadra** tu línea temporal con `horas-ataque.txt`. Si algo no cuadra, explica por qué.

Después redacta el informe del apartado 8. El resumen tiene que entenderlo alguien que no sabe qué es una inyección SQL. En erradicación y recuperación, apunta lo que hay que arreglar en la web: cada alerta del simulacro corresponde a un fallo del código de miweb, y el martes 24, en PPS, los vas a corregir. Hoy no los arregles: el jueves, en HE, vas a atacar la misma web.

11. Antes de irte: deja la web como estaba

El jueves, en HE, tu Kali tiene que poder llegar a tu web. Quita la línea `deny` de la Kali, comprueba y recarga:

```
sudo cp ~/ic/simulacro/nginx-miweb.antes /etc/nginx/sites-available/miweb
sudo nginx -t
sudo systemctl reload nginx
```

Comprueba desde la Kali que vuelve a responder 200. Las reglas de miweb y las dos fuentes del agente se quedan: así, todo lo que hagas el jueves quedará registrado en tu Wazuh y lo investigarás el miércoles que viene.

12. Si algo no funciona

La Kali no llega a `192.168.56.1:8080`: con `ip -br a` comprueba que la Kali tiene una dirección 192.168.56.1xx. Si no la tiene, apágala y vuelve a pasar `red-solo-anfitrion.sh` de la práctica de PPS. Si la tiene, prueba desde el Ubuntu `curl http://192.168.56.1:8080/salud`; si ahí sale 403, la configuración de nginx no tiene la línea `allow 192.168.56.0/24`.

`curl` desde el Ubuntu da `Connection refused`: nginx no está en marcha (`sudo systemctl status nginx`). Si responde 502 `Bad Gateway`, nginx funciona pero miweb no: `sudo journalctl -u miweb -n 30 --no-pager`.

`wazuh-logcollector -t` da error de XML: revisa que has pegado los bloques dentro de un `<ossec_config>` y que no ha quedado ningún comentario `<!--` sin su `-->`. Si no lo ves, vuelve a la copia `ossec.conf.antes-miweb`.

`grep miweb /var/ossec/logs/ossec.log` no saca nada después de reiniciar el agente: no ha leído tu configuración nueva. Comprueba con `sudo grep -c localfile /var/ossec/etc/ossec.conf` que tus bloques están y que el agente se ha reiniciado (`systemctl status wazuh-agent`).

`subir-reglas.sh` termina en `ERROR`: el mensaje dice el fichero y la línea. Si has tocado `miweb_rules.xml`, vuelve a descomprimir el original. Para quitar las reglas del manager: `./subir-reglas.sh quitar`.

En `wazuh-logtest` la línea JSON se decodifica (`name: 'json'`) pero en la fase 3 no sale ninguna regla: tus reglas no están cargadas en esa sesión, o la línea que has pegado no es de miweb (mira que lleve `"app": "miweb"`). Cierra `wazuh-logtest` y ábrelo otra vez después de `subir-reglas.sh`.

Lanzas el ataque y no sale ninguna alerta de miweb: comprueba que llegan eventos (`sudo tail /var/log/miweb/seguridad.log`), que el agente está `Active` y que reiniciaste el manager después de subir las reglas. Si lanzaste el ataque justo después de reiniciar, vuelve a lanzarlo: con un minuto de espera basta.

Sale la 100201 pero nunca la 100203: tiene que haber seis fallos desde la misma IP en menos de dos minutos. Si has puesto mucha pausa entre contraseñas, no se cumple.

Plan B, si la Kali no llega a tu web. Lanza el ataque desde el propio Ubuntu:

```
OBJETIVO=http://127.0.0.1:8080 bash ataque-simulado.sh
```

Funciona igual, pero en los registros la IP atacante será `127.0.0.1`, la del propio servidor. Las alertas saltan, pero la contención por IP no se puede hacer (te bloquearías a ti). En ese caso, en el apartado 8 escribe la línea `deny` que pondrías, explica por qué no la aplicas y qué harías en su lugar. Apunta en la plantilla qué le pasa a tu Kali: arreglarlo es parte del itinerario de recuperación.

Plan B, si tu Wazuh no arranca. Haz los apartados 6 a 11 solo con los registros: el triaje lo haces con `jq` sobre `seguridad.log` (por ejemplo, `jq -r '[.ts, .evento, .usuario, .ip] | @tsv' /var/log/miweb/seguridad.log`) y con `grep` sobre el registro de nginx. Para probar las reglas, usa el manager suelto `ic-logtest` de la semana pasada con `CONTENEDOR=ic-logtest ./subir-reglas.sh`.

13. Elige tu itinerario

Recuperación

Si no tienes miweb desplegada o tu Kali no llega a ella, dedica la primera hora a eso con la práctica de PPS del 17/11, y después haz esta hoja con el plan B que te haga falta. Lo mínimo es tener las reglas cargadas, el ataque lanzado (desde la Kali o desde el Ubuntu), el triaje del apartado 7 y las evidencias del apartado 9.

Ampliación

Elige una:

1. **El registro de errores de nginx.** Añade un tercer bloque `<localfile>` para `/var/log/nginx/miweb_error.log` y comprueba con `wazuh-logtest` qué decodificador y qué regla le aplica Wazuh a la línea `access forbidden by rule`. Escribe una regla 100213, hija de esa, que suba el nivel cuando una IP bloqueada insiste más de 5 veces en un minuto. Pruébala desde la Kali con la contención puesta.
2. **Cuentas nuevas sospechosas.** Escribe una regla 100214 que salte con nivel 9 cuando la misma IP crea más de 3 cuentas en 5 minutos. Recuerda lo que pasa con el nivel de la regla de la que cuelga la correlación. Pruébala creando cuentas desde la Kali con `curl -d nombre=... -d clave=... http://192.168.56.1:8080/registro`.

14. Preguntas

1. ¿Por qué el registro de miweb no necesita decodificador propio y el del portal de nóminas de la semana pasada sí?
2. El 401 de un acceso fallido aparece en las dos fuentes. ¿Qué sabe la aplicación que no sabe nginx? ¿Y al revés?
3. La 100203 cuenta fallos desde la misma IP sin mirar el usuario. Pon un ejemplo de ataque que no detectaría y uno en el que saltaría sin que hubiera ataque.
4. ¿Qué le pasaría a la 100211 si la 100210 tuviera nivel 0? ¿Cómo lo has comprobado?
5. ¿Qué alerta te hace pasar de "nos están atacando" a "nos han atacado con éxito"? ¿Por qué?
6. Después de la contención, ¿qué reglas dejan de ver al atacante? ¿Es eso un problema?
7. ¿Para qué sirve calcular el SHA-256 de las evidencias si nadie las va a tocar?

15. Qué tienes que entregar

Una carpeta `NN-IC-simulacro` comprimida en ZIP con:

1. `plantilla-incidente.md` rellena: punto de partida, reglas, horas del ataque, triaje, contención, evidencias, línea temporal, informe y respuestas.
2. La carpeta `evidencias/` con las copias, `SHA256SUMS` y la salida de `sha256sum -c`.
3. Los bloques `<localfile>` que has añadido a tu agente (copia solo esa parte de `ossec.conf`).
4. La captura de la alerta 100204 en el Dashboard o, si no has podido, qué has comprobado y dónde te has quedado.

Si no terminas, entrega lo que tengas e indica en qué punto te has quedado.

16. Antes de entregar, comprueba que

1. `sha256sum -c SHA256SUMS` da `OK` en todas las copias.
2. La tabla de triaje tiene gravedad y decisión para cada tipo de alerta, no solo el nivel.
3. La línea temporal tiene hora local y UTC, agrupa los sucesos y cuadra con `horas-ataque.txt`.
4. Has quitado el bloqueo de la Kali y la Kali vuelve a llegar a tu web.
5. Tu manager y tu agente siguen funcionando.
6. Ningún fichero lleva contraseñas de verdad, tokens ni datos de compañeros. Las de los usuarios de ejemplo de miweb son inventadas.

17. Seguridad

El ataque de hoy es real, aunque lo lance un script: inyección SQL, fuerza bruta y acceso a datos ajenos. Solo se lanza desde tu Kali contra tu propia web, en tu ordenador. Nunca contra la web de un compañero, ni aunque te deje: para eso hace falta un alcance por escrito, como en HE. nginx responde 403 a cualquier equipo que no sea el tuyo, y no lo cambies.

Los registros de hoy contienen usuarios, IP y, en el caso de la búsqueda, lo que el atacante ha escrito. En un sistema real serían datos personales. Trátalos como tales: se entregan en la carpeta de la práctica y no se publican en ningún repositorio.

Referencias

- NIST SP 800-61 Rev. 3, Incident Response Recommendations and Considerations for Cybersecurity Risk Management: https://csrc.nist.gov/pubs/sp/800/61/r3/final
- Wazuh, recogida de registros con `localfile`: https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/localfile.html
- Wazuh, reglas y decodificadores propios: https://documentation.wazuh.com/current/user-manual/ruleset/custom.html
- Wazuh, sintaxis de reglas (`if_matched_sid`, `same_field`, `frequency`): https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/rules.html
- nginx, módulo `ngx_http_access_module` (`allow` y `deny`): https://nginx.org/en/docs/http/ngx_http_access_module.html

- MITRE ATT&CK, T1110 Brute Force: https://attack.mitre.org/techniques/T1110/
- MITRE ATT&CK, T1190 Exploit Public-Facing Application: https://attack.mitre.org/techniques/T1190/
- OWASP, SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection